Work Requirement 3 - REST Principles Stress Test

This work requirement will assess API behaviors and design claims against core RESTful principles. The focus is on evaluating whether given statements and scenarios are consistent with core REST design principles, and on reasoning about the consequences of design choices. No coding or API implementation.

Objectives for Work Requirement 3

- Identify REST principles in described API behaviors
- Assess whether design choices are consistent with core REST design principles
- Reason about the consequences of violating REST principles
- Explain architectural trade-offs clearly and precisely

Case 1 *"The API uses URLs such as /createOrder and /getUserData, but otherwise uses correct HTTP methods." Answer the following:*

Which REST principle is involved:

The principles involved for REST (Representational State Transfer) for this case uniform interface, and resource structured design. The REST principle is about treating every piece of data as a resource, where each of the resources is identified by a unique URL. Thereby, the standard HTTP methods such as GET, POST, PUT and DELETE are used to interact with resources in a predictable and consistent way (ONF, Module 4).

Whether the design aligns with that principle:

In this case the design does not properly align with the REST principles. This is because the URLs describe actions (verbs) rather than the resources (nouns). A resource is an entity that can be named and manipulated, which is identified through a unique URL. For example `createOrder` and `getUserData` describe the action performed instead of describing the resource that is worked with. This approach mixes actions into the URL path, rather than separating what the resource is from how it is performed (ONF, Module 4).

One concrete problem this design can cause:

One concrete problem is that the API is less predictable and can become difficult for developers to interpret and handle consistently because the URL describes actions rather than resources. Therefore, this does not clearly separate the resource from the action in this case (ONF, Module 4).

Who is affected by this problem:

In this case the developers, maintainers, and clients may be affected due to URL structure. The structure makes the API less predictable and difficult to interpret, extend and maintain (ONF, Module 4).

Case 2: *"The server stores session data so it knows which user is logged in, even if no authentication token is sent with the request."Answer the following:*

Which REST principle is involved:

For this case, the principle of statelessness is involved. Each request sent to the server has to have all the necessary information for the server to process it. However, the server does not need to store information about the client's state between requests (ONF, Module 4).

Whether the design aligns with that principle:

This design does not align with the statelessness principle. The server relies on stored session data to identify the logged in user, rather than receiving the necessary information with each request. This breaks the advantages of statelessness and creates a dependency on the session state/data to know which user is logged in (ONF, Module 4).

One concrete problem this design can cause:
One concrete problem in this case is that the server must handle session data of which user is logged in. The dependency on session state would make it difficult to distribute the request across servers if the session data is not shared between them (ONF, Module 4).

Who is affected by this problem:
I believe both the server and developers are affected, as this approach can make the system difficult to scale and maintain. If session data is not shared between the servers, the requests might need to be redirected to the server storing the session, making it difficult to distribute requests independently across servers (ONF, Module 4).

Case 3: "The API returns different response formats for different endpoints, depending on which team implemented them." Answer the following:

Which REST principle is involved:
This case involves the uniform interface principle, which encourages consistent conventions when various clients interact with the resources in the API. Keeping consistent response formats, status codes, media types, and HTTP methods ensures the API is predictable and intuitive, regardless of the platform or languages (ONF, Module 4).

Whether the design aligns with that principle:
The design does not align with the uniform interface principle because different endpoints have different response formats. This breaks the consistency that makes the API predictable and intuitive. As a result, developers cannot rely on the same conventions when they interact with the different endpoints (ONF, Module 4).

One concrete problem this design can cause:
One concrete problem with this case is the increased complexity with integration. Developers may need to adapt to the different response formats from the different endpoints, rather than following the consistent approach in the uniform interface principle (ONF, Module 4).

Who is affected by this problem:
I believe the developers are affected by this problem due to integration and maintenance. As mentioned in the previous question, they may have to adapt to the different endpoints and their response formats. Further, maintenance also would become more complex if several teams continued developing the API without a coherent implementation strategy (ONF, Module 4).

Case 4: *"If an API uses HTTP and JSON, it is RESTful."* Answer the following:

Whether you agree or disagree:

I do not fully agree with the statement, although HTTP and JSON are important for building REST APIs. I disagree with the statement that an API automatically becomes RESTful because it uses HTTP and JSON. An API should align with the core principles for REST to be considered RESTful (ONF, Module 4).

Which REST principles support your reasoning:

The reason I do not fully agree is that RESTful API involves more than just HTTP and JSON. REST emphasizes resource oriented design, statelessness and a uniform interface, thereby following certain principles to make APIs RESTful. Relying solely on HTTP and JSON does not guarantee that these principles are followed. Therefore, the claim that these factors make it RESTful is not fully true (ONF, Module 4).

One consequence of believing this claim in practice:

In practice, developers might believe that an API follows REST because it uses HTTP and JSON. As a result, they might ignore important design principles for REST APIs, which might cause unnecessary complexity and inconsistency. As seen e.g. in Case 3, inconsistencies in the response formats between the endpoints could make the API difficult to integrate and maintain. This example demonstrates that these factors alone do not guarantee consistent and predictable REST APIs (ONF, Module 4).

Case 5: *"Statelessness mainly benefits the server, not the client."*Answer the following

Whether you agree or disagree:

I disagree with this statement because I believe statelessness benefits both the server and client. The server benefits from reduced dependency on session data, and only needs the necessary information from the client to process the request. Therefore the server does not need to store and handle session data between requests. As a result the client is responsible for providing the necessary information with each request, for example authentication tokens in the header and pagination parameters. I would believe both benefit from the predictable and independent communication between the server and client (ONF, Module 4).

Which REST principles are involved:

In this statement the Statelessness principle in REST is involved, which also supports scalability by enabling requests to be handled independently (ONF, Module 4).

One practical effect on system design:

A practical effect of Statelessness is that the system can scale easily and handle requests independently. If any server can handle requests independently without knowing what happened before, the system can scale easily (ONF, Module 4). An example is when traffic

and demand increase e.g. during holidays, the requests can be distributed across multiple servers to handle the incoming traffic.

Case 6: *"A uniform interface is mostly about naming conventions."* Answer the following:

Whether you agree or disagree:
I do not fully agree with this statement. This is because a uniform interface involves more than naming conventions. Clear resource-oriented URLs do support a consistent API design. However, the uniform interface principle also includes consistent use of HTTP methods, status codes, media types, and clear links between resources. These conventions are important to ensure the API can be interpreted by developers and integrated consistently (ONF, Module 4).

Which REST principles are involved:
The REST principle involved is the uniform interface principle, which ensures that resources in an API are handled consistently. The principle is to follow consistent use of HTTP methods, standardized status codes for success and errors, uniform media types, and clear links between resources. These conventions help make APIs self descriptive, and enable developers to interpret how to use them by exploring their structure (ONF, Module 4).

One concrete example that supports your reasoning:
One concrete example is an API that uses clear naming conventions, but still returns different response formats across different endpoints, as seen in Case 3. Therefore, even with clear and consistent naming conventions, the API can become unpredictable and difficult to integrate (ONF, Module 4).

Resources

ONF Module 4:

- 4.1 HTTP, HTTPS, and REST - A Quick Refresher
- 4.2 Core Principles of RESTful API Design